

Modelagem e Validação de um Protocolo R-UDP por Simulação de Eventos Discretos utilizando SimPy

Nara Raquel Dias Andrade

*Programa de Pós-Graduação em Ciência da Computação
Universidade Federal do Piauí – UFPI*

Picos, Brasil

nara.andrade@ufpi.edu.br

Abstract—Protocolos baseados em UDP oferecem baixa sobrecarga e elevada eficiência para transferência de dados, porém não fornecem mecanismos nativos de confiabilidade, como confirmação de recebimento, retransmissão de pacotes e controle de sequência. Na primeira etapa deste trabalho foi implementado um protocolo Reliable UDP (R-UDP) utilizando o algoritmo Go-Back-N e sua avaliação foi realizada experimentalmente em diferentes condições de rede. Nesta segunda etapa é apresentada a modelagem estocástica desse protocolo por meio de um simulador de eventos discretos desenvolvido em SimPy, permitindo reproduzir seu comportamento sem a necessidade de execução em ambiente real. O simulador incorpora modelos probabilísticos para atraso, perda de pacotes, retransmissões e confirmação de recebimento, cujos parâmetros foram obtidos a partir dos experimentos da implementação real. Foram realizados experimentos de validação comparando os resultados simulados com aqueles observados na Fase 1, além de análises de sensibilidade envolvendo diferentes níveis de jitter, tamanhos de janela, tamanhos de arquivo e cenários de estresse. Os resultados demonstram que o modelo reproduz adequadamente as tendências observadas na implementação real, constituindo uma ferramenta útil para avaliação de desempenho e análise de protocolos confiáveis sobre UDP.

Reliable UDP, Simulação por Eventos Discretos, SimPy, Go-Back-N, Modelagem Estocástica, Redes de Computadores.

I. INTRODUÇÃO

Protocolos de transporte desempenham papel fundamental nas redes de computadores ao prover mecanismos para comunicação confiável entre aplicações distribuídas [1], [2]. Enquanto o Transmission Control Protocol (TCP) incorpora mecanismos de controle de fluxo, retransmissão e ordenação dos dados, o User Datagram Protocol (UDP) fornece um serviço de comunicação sem conexão, priorizando simplicidade e baixa sobrecarga [1], [3], [4] e reduzida latência, porém sem qualquer garantia de entrega ou confiabilidade.

Em diversas aplicações, como sistemas de monitoramento, jogos em tempo real e aplicações multimídia, torna-se desejável combinar a eficiência do UDP com mecanismos adicionais de confiabilidade implementados em nível de aplicação [1]. Nesse contexto, protocolos do tipo *Reliable UDP* (R-UDP) permitem incorporar funcionalidades como numeração

de sequência, confirmações (ACKs), temporizadores e retransmissões, preservando a simplicidade do protocolo subjacente.

Na primeira etapa deste projeto foi desenvolvido um protocolo R-UDP baseado no algoritmo Go-Back-N, cuja implementação foi avaliada experimentalmente por meio de testes realizados em ambiente Docker sob diferentes condições de atraso e perda de pacotes. Os experimentos permitiram caracterizar o comportamento do protocolo em cenários controlados, fornecendo métricas como tempo de transferência, throughput, retransmissões e utilização da rede.

Embora experimentos reais forneçam resultados representativos, sua execução demanda infraestrutura computacional, tempo de processamento e repetidas execuções para obtenção de resultados estatisticamente confiáveis. Nesse cenário, a simulação por eventos discretos surge como uma alternativa capaz de reproduzir o comportamento do sistema com menor custo computacional, além de possibilitar a avaliação de novos cenários que seriam mais difíceis de implementar em ambiente real.

Este trabalho apresenta a modelagem estocástica do protocolo R-UDP desenvolvido na etapa anterior por meio de um simulador implementado utilizando a biblioteca SimPy. O modelo incorpora distribuições probabilísticas para atraso, perdas de pacotes, retransmissões e confirmações de recebimento, cujos parâmetros foram definidos a partir dos experimentos conduzidos na implementação real. Além da validação do simulador frente aos resultados experimentais, são realizados estudos de sensibilidade envolvendo diferentes níveis de jitter, tamanhos de janela de transmissão, tamanhos de arquivo e cenários de maior taxa de perda, permitindo avaliar o comportamento do protocolo em condições distintas daquelas originalmente experimentadas.

Como principais contribuições deste trabalho destacam-se: (i) o desenvolvimento de um modelo de simulação por eventos discretos para um protocolo R-UDP baseado em Go-Back-N; (ii) a validação do modelo por comparação com dados experimentais obtidos na implementação real; e (iii) a realização de análises complementares que demonstram o impacto de diferentes parâmetros de rede sobre o desempenho do protocolo.

O restante deste artigo está organizado da seguinte forma. A Seção II apresenta os conceitos teóricos utilizados no desenvolvimento do trabalho. A Seção III descreve a metodologia empregada na construção do simulador e na realização dos experimentos. A Seção IV apresenta os resultados obtidos e sua análise. Por fim, a Seção VII apresenta as conclusões e perspectivas para trabalhos futuros.

II. FUNDAMENTAÇÃO TEÓRICA

A. Simulação por Eventos Discretos

A simulação por eventos discretos consiste em uma técnica de modelagem utilizada para representar sistemas dinâmicos cuja evolução ocorre em instantes discretos do tempo. Nesse paradigma, o estado do sistema é alterado apenas quando um evento ocorre, como o envio de um pacote, a chegada de uma confirmação de recebimento (ACK) ou a expiração de um temporizador. Essa abordagem reduz significativamente o custo computacional quando comparada a simulações baseadas em passos temporais fixos, tornando-se amplamente utilizada na avaliação de desempenho de sistemas computacionais [6], [7], redes de comunicação e processos industriais.

No presente trabalho foi utilizada a biblioteca SimPy [8], desenvolvida em Python, que fornece mecanismos para construção de simuladores orientados a processos. Cada entidade do sistema, como emissor, receptor e canal de comunicação, é representada por processos independentes que interagem por meio de eventos agendados em um relógio virtual compartilhado. Dessa forma, torna-se possível reproduzir o comportamento temporal do protocolo R-UDP sem a necessidade de utilização de infraestrutura física ou de comunicação real entre máquinas.

B. Protocolo Go-Back-N

O protocolo Go-Back-N pertence à classe dos protocolos de Janela Deslizante (*Sliding Window*) e constitui um mecanismo clássico de controle de erros utilizado em protocolos confiáveis de transporte de dados [1], [2]. Seu funcionamento baseia-se na transmissão contínua de múltiplos pacotes antes da recepção de suas respectivas confirmações, permitindo melhor utilização do enlace quando comparado ao protocolo Stop-and-Wait.

No Go-Back-N, o emissor mantém uma janela contendo os pacotes que podem ser transmitidos simultaneamente. Caso um pacote seja perdido ou um temporizador expire antes da chegada do ACK correspondente, o protocolo retransmite todos os pacotes a partir daquele que originou a falha. Embora esse mecanismo possa gerar retransmissões adicionais em cenários com elevada perda de pacotes, sua implementação apresenta menor complexidade quando comparada a protocolos como o Selective Repeat.

Na implementação desenvolvida neste trabalho, o protocolo R-UDP utiliza numeração sequencial de pacotes, confirmações cumulativas, temporizadores de retransmissão e controle de janela baseado no algoritmo Go-Back-N, reproduzindo os mecanismos fundamentais empregados em protocolos confiáveis de transporte [5].

C. Modelagem Estocástica de Redes

A modelagem estocástica busca representar sistemas reais por meio de variáveis aleatórias e distribuições probabilísticas capazes de reproduzir o comportamento observado experimentalmente. Em redes de computadores, características como atraso de propagação, perda de pacotes e variações temporais da comunicação apresentam natureza probabilística, tornando inadequada sua representação por valores determinísticos.

Neste trabalho, a latência da rede foi modelada por meio de uma distribuição Normal, cuja média corresponde ao atraso configurado em cada cenário experimental e cujo desvio padrão representa o nível de jitter introduzido na comunicação. A perda de pacotes foi representada por um processo de Bernoulli [9], no qual cada transmissão possui uma probabilidade fixa de descarte independente das demais.

Além da modelagem probabilística do canal de comunicação, o simulador incorpora eventos associados ao envio de pacotes, recebimento de ACKs, retransmissões e expiração de temporizadores. Essa abordagem permite reproduzir, de forma simplificada, os principais fenômenos observados durante os experimentos realizados na implementação real, possibilitando tanto a validação do modelo quanto a exploração de novos cenários de rede sem necessidade de experimentação em ambiente físico.

III. METODOLOGIA

O desenvolvimento deste trabalho foi realizado em duas etapas complementares. Na primeira etapa, descrita no trabalho anterior, foi implementado um protocolo Reliable UDP (R-UDP) utilizando sockets em Python e executado em ambiente Docker sob diferentes condições de rede. Os resultados experimentais dessa implementação serviram como base para a construção do modelo de simulação desenvolvido nesta segunda etapa.

A metodologia adotada teve como objetivo construir um simulador por eventos discretos capaz de reproduzir o comportamento observado na implementação real, preservando os principais mecanismos do protocolo e permitindo a realização de novos experimentos de forma controlada.

A. Arquitetura do Simulador

O simulador foi desenvolvido utilizando a biblioteca SimPy [8], que implementa o paradigma de simulação por eventos discretos. Nesse modelo, o tempo é representado por um relógio virtual, sendo atualizado apenas quando ocorre algum evento relevante para o sistema.

A arquitetura do simulador foi organizada em componentes independentes responsáveis pelo emissor, receptor, canal de comunicação e coleta das métricas experimentais. Cada componente é executado como um processo independente dentro do ambiente de simulação, permitindo reproduzir o fluxo completo de transmissão do protocolo R-UDP.

A Figura 1 apresenta a arquitetura geral do simulador desenvolvido.

A Figura 2 detalha o fluxo de eventos do protocolo simulado. O emissor transmite pacotes dentro da janela de

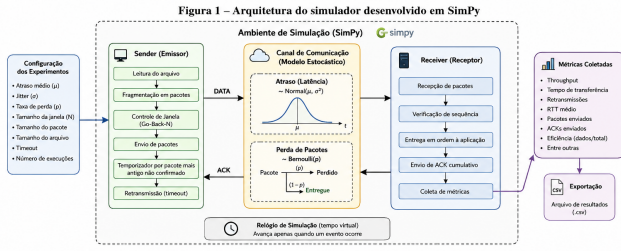


Fig. 1. Arquitetura geral do simulador desenvolvido em SimPy.

transmissão, o canal aplica atraso e perda probabilística, o receptor verifica a sequência dos pacotes recebidos e envia confirmações cumulativas. Caso o temporizador expire antes da chegada do ACK correspondente, ocorre retransmissão a partir da base da janela, conforme o comportamento do algoritmo Go-Back-N.

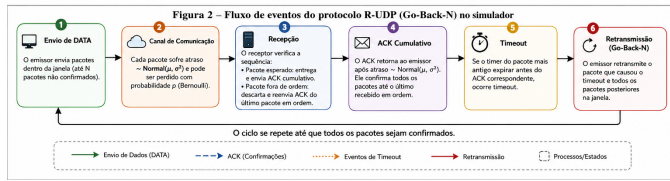


Fig. 2. Fluxo de eventos do protocolo R-UDP baseado em Go-Back-N no simulador.

O emissor realiza a fragmentação do arquivo em pacotes de tamanho fixo e controla a transmissão utilizando o algoritmo Go-Back-N. O canal de comunicação aplica os modelos probabilísticos responsáveis pelo atraso e pela perda de pacotes. O receptor valida a sequência recebida, envia confirmações cumulativas e registra as métricas necessárias para análise posterior.

B. Modelagem Probabilística

A modelagem do canal de comunicação foi baseada nos resultados obtidos durante a implementação experimental da primeira etapa do projeto. O atraso de transmissão foi representado por uma distribuição Normal, cuja média corresponde ao atraso configurado em cada cenário experimental e cujo desvio padrão representa o nível de jitter introduzido na comunicação.

A perda de pacotes foi modelada utilizando uma distribuição de Bernoulli, na qual cada pacote possui uma probabilidade fixa de descarte independente das demais transmissões. Essa abordagem permitiu reproduzir os três cenários originalmente implementados com o comando `tc`, além de possibilitar a criação de novos cenários para avaliação do protocolo.

C. Configuração dos Experimentos

Todos os experimentos utilizaram arquivos fragmentados em blocos de 1024 bytes e o protocolo Go-Back-N como mecanismo de confiabilidade. Foram realizadas 30 execuções independentes para cada configuração experimental, permitindo

calcular médias, desvios padrão e intervalos de confiança de 95%.

Inicialmente foi realizada uma etapa de validação do simulador utilizando os mesmos cenários empregados na implementação real:

- Cenário A: atraso médio de 10 ms e perda de 0%;
- Cenário B: atraso médio de 50 ms e perda de 10%;
- Cenário C: atraso médio de 100 ms e perda de 20%.

Após a validação inicial, foram conduzidos experimentos adicionais para avaliar o impacto do jitter, da variação do tamanho da janela de transmissão, do tamanho do arquivo e de um cenário de estresse com taxa de perda de 25%. Em todos os experimentos foram registradas métricas como throughput, tempo de transferência, retransmissões, RTT médio e eficiência do protocolo.

D. Validação do Modelo

A validação do simulador foi realizada por meio da comparação entre as métricas obtidas na simulação e aquelas observadas durante a implementação experimental desenvolvida na primeira etapa do projeto. Foram comparados indicadores de desempenho como throughput, tempo de transferência, retransmissões e RTT médio, permitindo verificar a capacidade do modelo em reproduzir o comportamento do sistema real.

Além da comparação direta entre os resultados experimentais e simulados, foram realizadas análises de sensibilidade variando parâmetros do protocolo e do canal de comunicação. Essa abordagem permitiu avaliar a robustez do modelo e sua capacidade de representar diferentes condições de operação sem necessidade de experimentação em ambiente físico.

IV. RESULTADOS EXPERIMENTAIS

Esta seção apresenta os resultados obtidos com o simulador desenvolvido em SimPy. Inicialmente, é realizada a validação do modelo por comparação com os experimentos executados na implementação real do protocolo R-UDP. Em seguida, são apresentados os estudos de sensibilidade conduzidos para avaliar o impacto do jitter, do tamanho da janela de transmissão, do tamanho do arquivo e de cenários de maior perda de pacotes sobre o desempenho do protocolo.

A. Validação do Simulador

A primeira etapa consistiu em validar o modelo desenvolvido por meio da comparação entre os resultados obtidos na simulação e aqueles observados na implementação real do protocolo R-UDP. Foram comparadas métricas de desempenho como throughput, tempo médio de transferência e número médio de retransmissões para os três cenários de rede originalmente utilizados na Fase 1.

A Figura 3 apresenta a comparação do throughput médio entre a implementação real e o simulador. Como referência adicional, também são apresentados os resultados obtidos com o protocolo TCP.

Observa-se que o simulador reproduziu adequadamente a tendência de degradação do throughput à medida que as

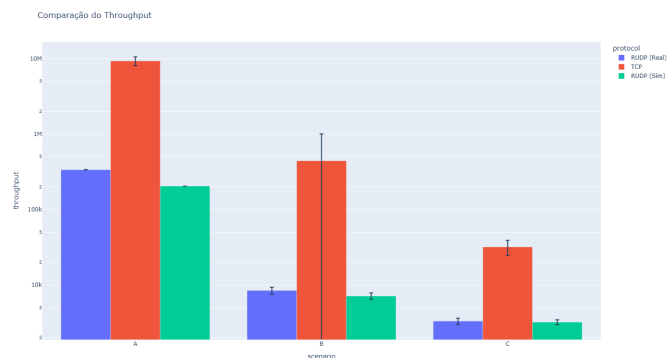


Fig. 3. Comparação do throughput médio entre a implementação real e o simulador.

condições da rede se tornaram mais severas. Embora exista diferença absoluta entre os valores observados no Cenário A, decorrente das simplificações adotadas no modelo de simulação, os cenários B e C apresentaram comportamento bastante próximo ao observado experimentalmente, indicando que o modelo captura de forma satisfatória os principais fenômenos envolvidos na transmissão confiável sobre UDP.

A Figura 4 apresenta exclusivamente a comparação entre os resultados do protocolo R-UDP implementado e aqueles produzidos pelo simulador.

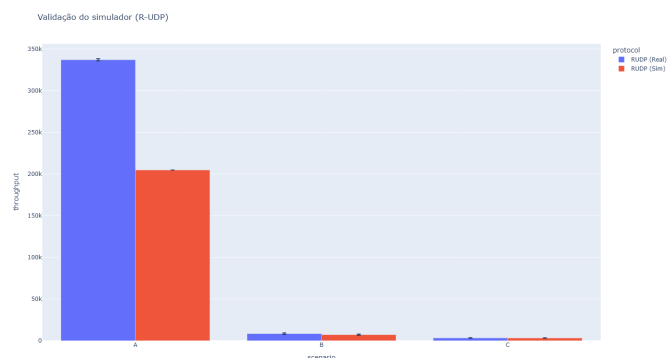


Fig. 4. Validação do modelo de simulação utilizando apenas os resultados do protocolo R-UDP.

Mesmo utilizando um modelo simplificado para representar o canal de comunicação, o simulador reproduziu o comportamento geral do protocolo, especialmente nos cenários com perda de pacotes, demonstrando sua capacidade de representar adequadamente o sistema real para fins de avaliação de desempenho.

B. Impacto do Jitter

Após a validação do modelo, foi realizado um estudo para avaliar o impacto do jitter sobre o desempenho do protocolo. Foram considerados diferentes desvios padrão para a distribuição Normal utilizada na modelagem do atraso, mantendo constantes os demais parâmetros da simulação.



Fig. 5. Influência do jitter sobre o RTT médio observado na simulação.

Os resultados mostram que o aumento do jitter produz crescimento gradual do RTT médio, sem provocar alterações significativas na taxa de retransmissões para os níveis avaliados. Esse comportamento indica que pequenas variações na latência afetam principalmente o tempo de propagação das mensagens, preservando a estabilidade geral do protocolo.

C. Influência do Tamanho da Janela de Transmissão

O protocolo Go-Back-N permite que múltiplos pacotes permaneçam simultaneamente em trânsito antes da recepção de suas confirmações. Dessa forma, o tamanho da janela de transmissão exerce influência direta sobre a utilização do enlace e, conseqüentemente, sobre o throughput do protocolo.

Para avaliar esse comportamento, foram realizados experimentos variando o tamanho da janela entre 1 e 32 pacotes, mantendo constantes os demais parâmetros da simulação.

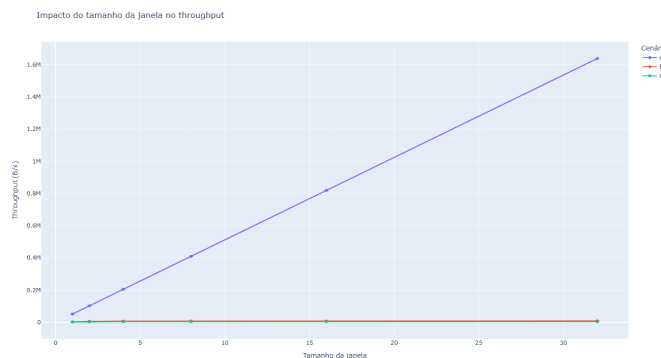


Fig. 6. Impacto do tamanho da janela sobre o throughput do protocolo.

Os resultados apresentados na Figura 6 mostram que o aumento da janela proporciona crescimento significativo do throughput no Cenário A, onde não há perdas de pacotes. Entretanto, nos cenários B e C, caracterizados por perdas de 10% e 20%, respectivamente, o ganho torna-se menos pronunciado devido ao aumento das retransmissões inerentes ao algoritmo Go-Back-N. Esse comportamento evidencia o compromisso existente entre maior paralelismo na transmissão e maior custo de recuperação de perdas.

D. Influência do Tamanho do Arquivo

Outra análise realizada consistiu em avaliar o comportamento do protocolo para diferentes tamanhos de arquivo, variando entre 1 MB e 100 MB.

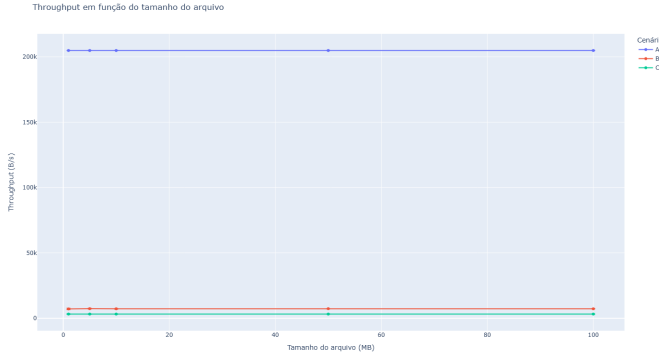


Fig. 7. Throughput médio em função do tamanho do arquivo transmitido.

Conforme ilustrado na Figura 7, o throughput permaneceu praticamente constante para todos os tamanhos de arquivo avaliados dentro de um mesmo cenário. Esse resultado indica que o desempenho do protocolo é predominantemente determinado pelas condições da rede e pelos mecanismos de confiabilidade empregados, apresentando baixa influência do volume total de dados transmitidos.

E. Cenário de Estresse

Após a validação do modelo e dos experimentos de sensibilidade, foi conduzido um cenário adicional de estresse, caracterizado por taxa de perda de 25%, superior àquela utilizada durante a implementação experimental da Fase 1.

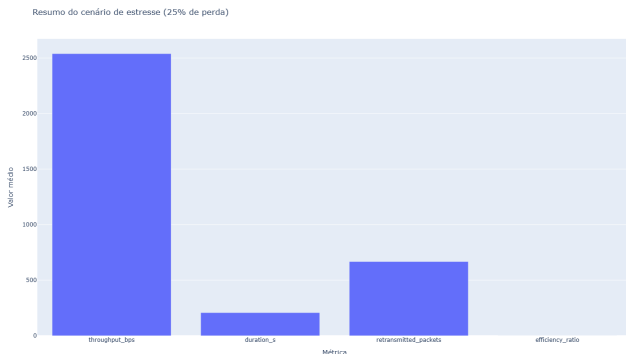


Fig. 8. Resumo das métricas obtidas no cenário de estresse com perda de 25%.

Os resultados demonstram redução do throughput, aumento do tempo total de transferência e crescimento expressivo do número de retransmissões. Apesar da degradação de desempenho esperada, o protocolo foi capaz de concluir todas as transmissões simuladas, indicando que os mecanismos de confiabilidade implementados permanecem operacionais mesmo sob condições significativamente mais severas do que aquelas utilizadas durante a etapa experimental.

F. Eficiência do Protocolo

Além das métricas tradicionais de desempenho, foi analisada a eficiência do protocolo, definida como a razão entre o número de pacotes úteis transmitidos e o total de pacotes enviados, incluindo retransmissões e confirmações (ACKs).

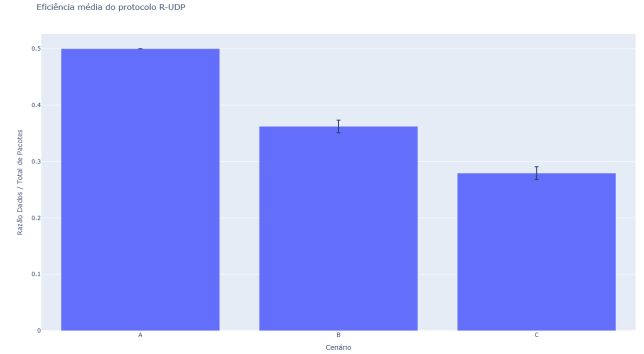


Fig. 9. Eficiência média do protocolo R-UDP nos diferentes cenários de rede.

Observa-se que a eficiência diminui progressivamente à medida que aumentam as perdas de pacotes. Esse comportamento decorre do maior número de retransmissões exigidas pelo algoritmo Go-Back-N para garantir a entrega confiável dos dados. Mesmo assim, o protocolo mantém capacidade de completar as transmissões, evidenciando que o custo adicional de confiabilidade cresce de forma compatível com o aumento da severidade do canal de comunicação.

V. DISCUSSÃO

VI. DISCUSSÃO

Os resultados obtidos demonstram que o modelo desenvolvido foi capaz de reproduzir o comportamento geral observado na implementação experimental do protocolo R-UDP. Embora diferenças quantitativas tenham sido observadas entre os valores absolutos obtidos na simulação e aqueles medidos durante os experimentos reais, principalmente no cenário com ausência de perdas, o simulador reproduziu corretamente as tendências de degradação de desempenho à medida que aumentaram o atraso e a taxa de perda de pacotes.

A principal divergência foi observada no Cenário A, no qual o throughput obtido pela implementação real foi superior ao produzido pelo simulador. Essa diferença pode ser atribuída às simplificações adotadas durante a modelagem. O simulador considera apenas os principais eventos relacionados ao protocolo de transporte, como transmissão, perda, atraso, confirmações e retransmissões, não representando diversos aspectos presentes em um sistema operacional real, como escalonamento de processos, gerenciamento de buffers, otimizações da pilha de rede e sobrecargas inerentes ao ambiente Docker utilizado na etapa experimental. Nos cenários B e C, entretanto, verificou-se elevada concordância entre os resultados simulados e experimentais. Nessas condições, o impacto das perdas de pacotes e das retransmissões torna-se predominante sobre os demais fatores do sistema, fazendo com

que o modelo probabilístico adotado represente de maneira mais fiel o comportamento observado durante a implementação real.

Os experimentos de sensibilidade também forneceram informações importantes sobre o funcionamento do protocolo. A variação do jitter mostrou que pequenas flutuações no atraso aumentam o RTT médio, porém exercem influência limitada sobre o throughput quando comparadas aos efeitos provocados pela perda de pacotes. A análise do tamanho da janela evidenciou que janelas maiores proporcionam melhor utilização do enlace em condições favoráveis, enquanto seus benefícios diminuem à medida que cresce a taxa de perdas, em razão do aumento das retransmissões características do algoritmo Go-Back-N.

Da mesma forma, os experimentos envolvendo diferentes tamanhos de arquivo mostraram que o throughput permanece praticamente constante para cada cenário de rede, indicando que o desempenho do protocolo depende principalmente das características do canal de comunicação e não do volume total de dados transmitidos. O cenário de estresse confirmou essa tendência, demonstrando que, mesmo sob uma taxa de perda superior à utilizada na validação, o protocolo permaneceu funcional, embora com aumento significativo do tempo de transferência e redução da eficiência.

Em conjunto, esses resultados indicam que o simulador desenvolvido constitui uma representação consistente do protocolo R-UDP implementado na primeira etapa do projeto. Além de reproduzir adequadamente os fenômenos observados experimentalmente, o modelo permite investigar novos cenários de operação com menor custo computacional e maior flexibilidade, tornando-se uma ferramenta útil para estudos de desempenho de protocolos confiáveis sobre UDP.

VII. CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um simulador por eventos discretos para modelagem do protocolo Reliable UDP (R-UDP) implementado na primeira etapa do projeto. O simulador foi desenvolvido utilizando a biblioteca SimPy e reproduz os principais mecanismos do protocolo, incluindo transmissão por janela deslizante baseada em Go-Back-N, confirmações cumulativas, temporizadores, retransmissões, atraso de propagação e perdas probabilísticas de pacotes.

A validação do modelo foi realizada por meio da comparação entre os resultados obtidos na simulação e aqueles produzidos pela implementação real executada em ambiente Docker. As análises mostraram que o simulador reproduziu adequadamente as principais tendências observadas experimentalmente, especialmente nos cenários com perda de pacotes, apresentando comportamento consistente quanto ao throughput, tempo de transferência, retransmissões e eficiência do protocolo.

Além da etapa de validação, o simulador permitiu realizar estudos de sensibilidade envolvendo diferentes níveis de jitter, tamanhos de janela de transmissão, tamanhos de arquivo e um cenário de estresse com taxa de perda de 25%. Esses

experimentos demonstraram a flexibilidade da modelagem por eventos discretos para investigação do comportamento do protocolo em situações que demandariam maior esforço computacional e operacional caso fossem executadas exclusivamente em ambiente real.

Os resultados obtidos evidenciam que a utilização de simulação por eventos discretos constitui uma alternativa eficiente para análise de desempenho de protocolos confiáveis sobre UDP, permitindo reduzir o custo de experimentação sem perder a capacidade de representar os principais fenômenos observados na prática.

Como trabalhos futuros, pretende-se incorporar modelos mais sofisticados para representação do canal de comunicação, incluindo distribuições probabilísticas ajustadas a medições reais, perdas correlacionadas, múltiplos fluxos concorrentes e mecanismos alternativos de controle de erros, como o protocolo Selective Repeat. Essas extensões poderão ampliar a fidelidade do simulador e permitir sua aplicação em cenários de comunicação ainda mais complexos.

VIII. MATERIAIS COMPLEMENTARES

Com o objetivo de garantir a reprodutibilidade dos experimentos e facilitar a avaliação deste trabalho, o código-fonte, os dados experimentais e o vídeo técnico de demonstração foram disponibilizados nos seguintes endereços:

- **Repositório GitHub:**

https://github.com/NaraAndrad3/projeto_redes_rudp

- **Vídeo técnico de apresentação:**

https://youtu.be/SEU_VIDEO

O repositório contém todo o código-fonte desenvolvido nas duas etapas do projeto, incluindo a implementação do protocolo R-UDP, o simulador em SimPy, os scripts de execução dos experimentos, os arquivos de captura (PCAP), os conjuntos de dados utilizados nas análises, os notebooks desenvolvidos no Google Colab e o presente artigo em formato $\text{\LaTeX}$.

REFERENCES

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
- [2] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.
- [3] J. Postel, "User Datagram Protocol," RFC 768, 1980.
- [4] J. Postel, "Transmission Control Protocol," RFC 793, 1981.
- [5] V. Paxson, M. Allman and E. Blanton, "Computing TCP's Retransmission Timer," RFC 6298, 2011.
- [6] J. Banks, J. S. Carson, B. L. Nelson and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Pearson, 2010.
- [7] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015.
- [8] SimPy Development Team, "SimPy Documentation," Available: <https://simpy.readthedocs.io>
- [9] S. M. Ross, *Introduction to Probability Models*, 11th ed. Academic Press, 2014.